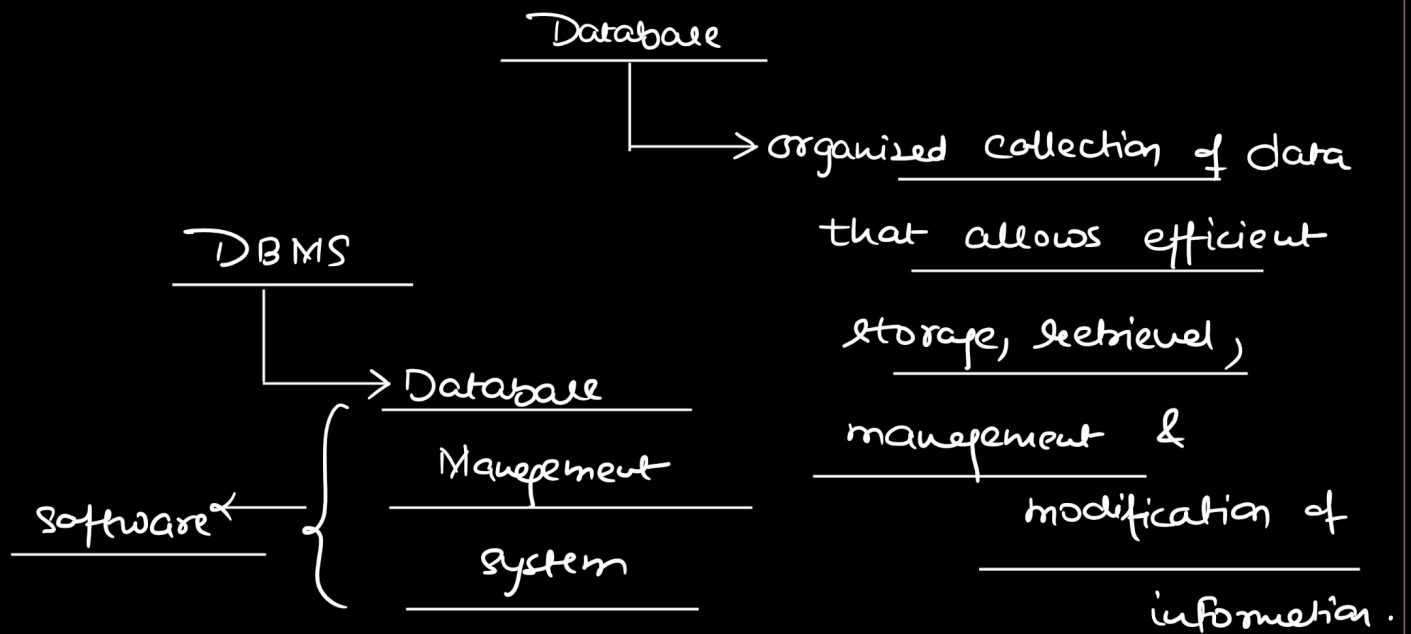




Importance of DBMS

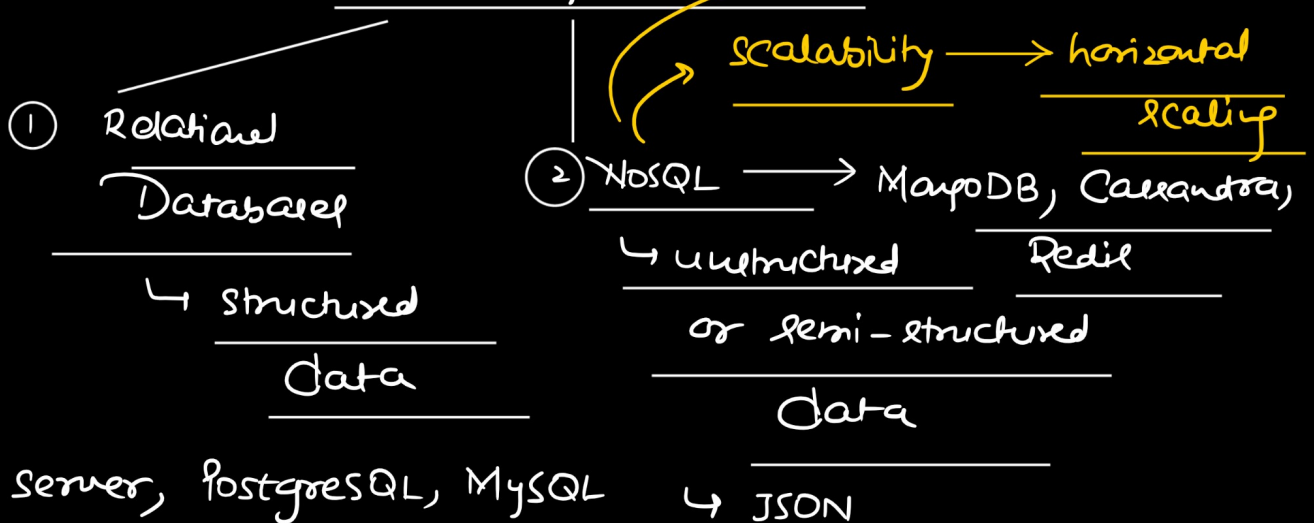
→ improve data sharing &
accessibility

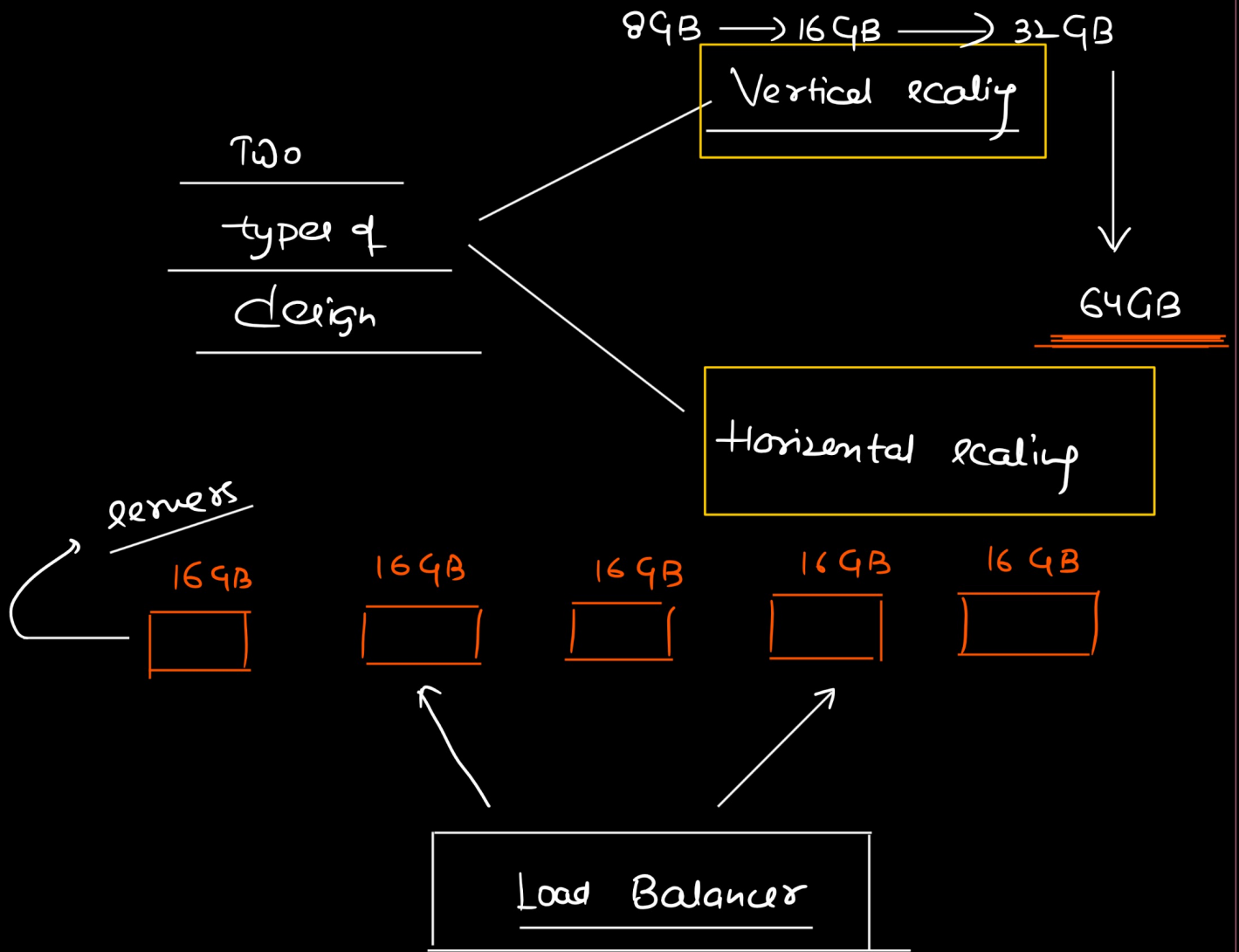
①

Horizontal scaling (scaling out) increases system capacity by adding more machines to a resource pool, ideal for distributed systems, high traffic, and fault tolerance. Vertical scaling (scaling up) upgrades an existing machine's CPU, RAM, or storage to enhance performance, which is easier to implement but has upper, cost-intensive limits

② Data Integrity & security

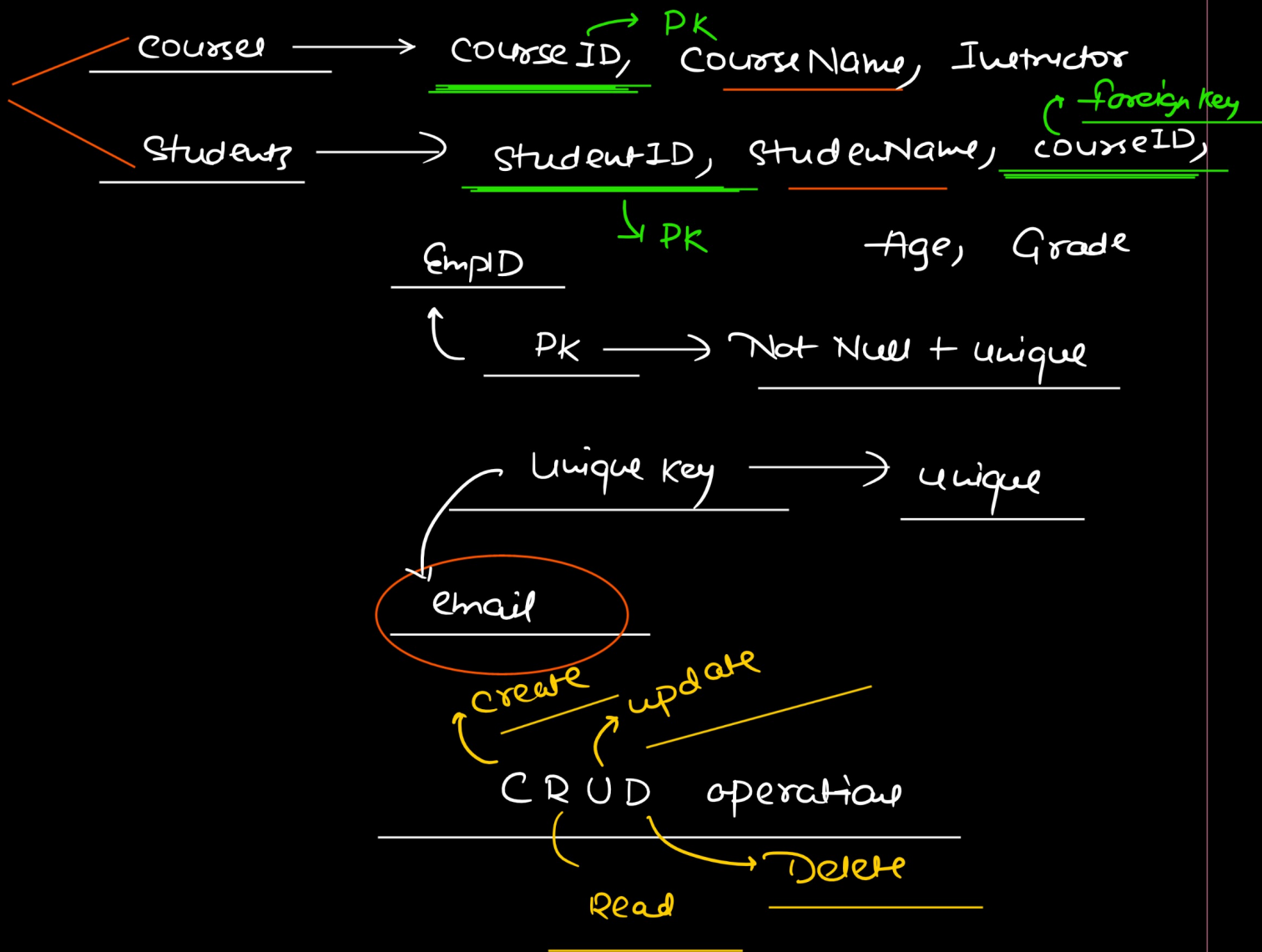
Type of DBMS





RDBMS → Table ⇒ 2-dimensional structure
made up of
rows & columns

Relationship ⇒ foreign key,
Primary key,
Unique key



Field	Type	Null	Key	Default	Extra
empID ✓	int	NO	PRI	<u>NULL</u>	auto_increment
firstName ✓	varchar(20)	NO		<u>NULL</u>	
lastName ✓	varchar(20)	NO		<u>NULL</u>	
Age ✓	int	NO		<u>NULL</u>	
Salary ✓	int	NO		<u>NULL</u>	
Location ✓	varchar(20)	NO		<u>NULL</u>	

→ schema of the table

SELECT * FROM employee
 WHERE Age < 29;

FROM → WHERE → SELECT
 filter

Course → CourseID → 1, 5, 7, 9
learners → selectedCourse → { 1 or 5 or 7 or 9 }
 (-foreign key)

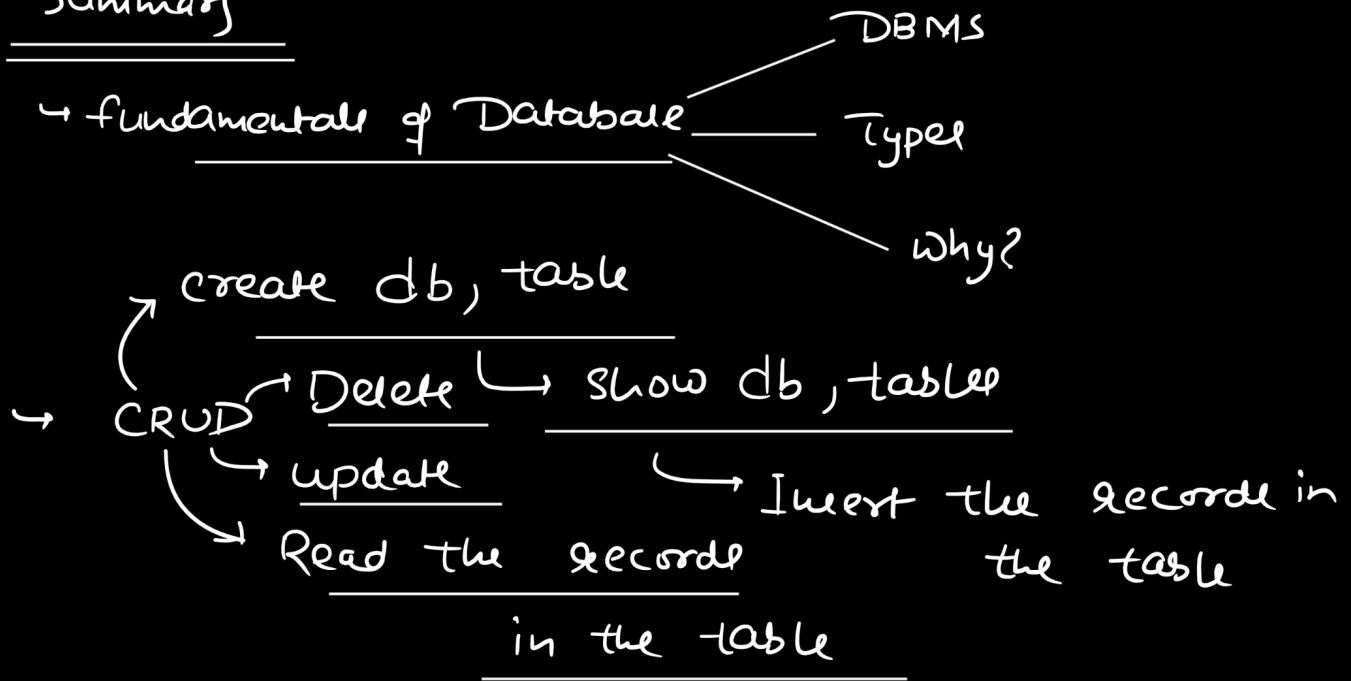
change in the records

update

change anything in schema

ALTER command

Summary



→ Alter → change in the scheme of the table

Two types

DML

(Data Manipulation Language)

manipulate the records

(update, delete, select)

DDL

(Data Definition Language)

scheme of the table

(Alter,

↳ filteration (WHERE)

↳ CSV, Excel file, JSON

↳ ORDER BY → sort the value, by default in an ascending order

LIMIT → limits

1st highest salary the output records

```
SELECT * FROM employee
WHERE Age > 25
ORDER BY salary DESC
LIMIT 1;
```

firstName, lastName

FROM → WHERE → SELECT
↓
ORDER BY
↓
LIMIT

The standard logical order of execution is:

FROM and JOINS: The database identifies the tables needed and combines their rows to form the initial dataset.

WHERE: Filters individual rows from the dataset based on specified conditions. This happens before any grouping or aggregation.

GROUP BY: Organizes the filtered rows into groups based on common values in specified columns.

HAVING: Filters the groups created by the GROUP BY clause. This is used for conditions involving aggregate functions, which are not permitted in the WHERE clause.

SELECT: Processes expressions (including column aliases and aggregate functions) and selects the columns to be included in the final result set. The DISTINCT keyword, if used, is also applied at this stage to remove duplicate rows.

ORDER BY: Sorts the final result set based on the specified columns, in ascending (ASC) or descending (DESC) order. Column aliases defined in the SELECT clause can be used here because the SELECT clause has already been executed.

LIMIT and OFFSET (or TOP): Restricts the number of rows returned, which is useful for pagination